

Python 문법 정리

네트워크 장비 점검 자동화 프로젝트에서 오늘 사용한 파일 처리, 리스트, 딕셔너리, 함수, import 구조

1. 오늘 만든 코드의 큰 흐름

이번 프로젝트는 running-config 파일을 읽고, N-01부터 N-05까지 점검 함수를 실행한 뒤, 결과를 Markdown 보고서로 저장하는 구조다.

```
network-config-auditor/  
main.py  
checks/  
cisco/  
n01_cisco.py  
n02_cisco.py  
n03_cisco.py  
n04_cisco.py  
n05_cisco.py  
reporters/  
markdown_report.py  
samples/  
running-config.txt  
reports/  
account_management_report.md
```

파일	역할	오늘 배운 문법 포인트
main.py	전체 실행 흐름	Path, import, 함수 호출, 리스트, 딕셔너리
n01~n05_cisco.py	항목별 점검 로직	함수, for문, if문, 리스트 append/extend
markdown_report.py	보고서 문자열 생성	딕셔너리 접근, 리스트 누적, join
running-config.txt	분석 대상 입력 파일	read_text로 읽는 텍스트 데이터

2. 파일 경로와 파일 읽기

파일 관련 경험이 적을 때 가장 헷갈리는 부분은 현재 실행 위치와 파일 위치다. 그래서 pathlib의 Path를 사용했다.

```
project_root = Path(__file__).resolve().parent  
config_file = project_root / "samples" / "running-config.txt"  
config_text = config_file.read_text(encoding="utf-8")
```

- Path(__file__)은 현재 실행 중인 Python 파일의 경로를 의미한다.
- resolve()는 상대 경로를 절대 경로로 바꿔준다.
- parent는 파일이 들어있는 폴더를 의미한다.
- / 연산자는 폴더와 파일 이름을 안전하게 이어 붙일 때 사용한다.
- read_text(encoding='utf-8')은 텍스트 파일 전체를 문자열로 읽는다.

3. 문자열을 줄 단위로 다루기

running-config는 긴 문자열 하나로 읽힌다. 하지만 설정 점검은 한 줄씩 검사해야 하므로 `splitlines()`를 사용했다.

```
for line in config_text.splitlines():
    stripped_line = line.strip()

    if stripped_line.startswith("username"):
        matched_lines.append(stripped_line)
```

- `splitlines()`는 여러 줄 문자열을 줄 단위 리스트로 바꾼다.
- `strip()`은 앞뒤 공백을 제거한다. Cisco 하위 설정 앞 공백을 비교할 때 유용하다.
- `startswith()`는 특정 키워드로 시작하는 설정 줄을 찾을 때 사용했다.

4. 리스트를 왜 썼는가

점검 결과는 한 개가 아니라 여러 줄이다. 그래서 확인된 설정, 미흡한 설정, 수동확인 항목을 리스트로 모았다.

```
found_settings = []
weak_settings = []
manual_settings = []

found_settings.append("security passwords min-length 8")
weak_settings.append("vty 패스워드 미설정")
manual_settings.extend(username_lines)
```

문법	의미	언제 사용했는지
<code>[]</code>	빈 리스트 생성	설정 결과를 여러 개 담기 위해 사용
<code>append(x)</code>	리스트에 값 하나 추가	미흡한 설정 문구 하나를 추가할 때
<code>extend(list)</code>	다른 리스트의 값을 펼쳐서 추가	찾은 설정 여러 줄을 한 번에 붙일 때
<code>if list:</code>	리스트가 비어있지 않은지 확인	미흡 항목이 있으면 취약 처리

주의: `extend('문자열')`처럼 문자열을 넣으면 글자 하나하나가 리스트에 들어간다. 문구 하나를 넣을 때는 `append()`를 쓰는 것이 맞다.

5. 딕셔너리를 왜 썼는가

각 점검 항목은 코드, 제목, 결과, 확인된 설정, 미흡한 설정, 수동확인 항목을 함께 가진다. 이런 이름 붙은 묶음에는 딕셔너리가 적합하다.

```
return {
    "code": code,
    "title": title,
    "result": result,
    "found": found_settings,
    "weak": weak_settings,
    "manual": manual_settings,
}
```

- 리스트는 순서 중심이다. 예: N-01, N-02, N-03 결과를 차례로 담기.

- 딕셔너리는 이름 중심이다. 예: result, found, weak처럼 의미 있는 이름으로 접근하기.
- 보고서 생성기에서는 `check('code')`, `check('found')`처럼 필요한 값을 꺼냈다.

6. 함수 반환값과 normalize_result

N-01~N-04는 result, found, weak 세 값을 반환한다. N-05는 수동확인 목록까지 필요해서 네 값을 반환한다. 그래서 main.py에서 반환 형태를 하나로 맞추는 normalize_result 함수를 만들었다.

```
def normalize_result(code, title, check_result):
    if len(check_result) == 3:
        result, found_settings, weak_settings = check_result
        manual_settings = {}
    else:
        result, found_settings, weak_settings, manual_settings = check_result
```

- len(check_result)는 반환된 값의 개수를 확인한다.
- N-05처럼 수동확인이 있으면 manual_settings도 받는다.
- 수동확인이 없는 항목은 빈 리스트 {}를 넣어 보고서 형식을 통일했다.

7. import와 파일 분리

main.py에 모든 점검 코드를 넣으면 너무 길어진다. 그래서 점검 항목별로 파일을 나누고 import로 불러왔다.

```
from checks.cisco.n01_cisco import check_n01_cisco
from checks.cisco.n02_cisco import check_n02_cisco
from reporters.markdown_report import make_markdown_report
```

- checks.cisco.n01_cisco는 checks/cisco/n01_cisco.py 파일을 의미한다.
- import 뒤의 check_n01_cisco는 그 파일 안에 있는 함수 이름이다.
- 이렇게 나누면 내일 N-06을 추가할 때 n06_cisco.py만 만들고 main.py에 한 줄 추가하면 된다.

8. checks 리스트 구조

반복되는 점검 실행을 줄이기 위해 main.py에서는 점검 항목 정보를 리스트로 만들었다.

```
checks = [
    ("N-01", "비밀번호 설정", check_n01_cisco),
    ("N-02", "비밀번호 복잡성 설정", check_n02_cisco),
    ("N-03", "암호화된 비밀번호 사용", check_n03_cisco),
]
```

```
for code, title, check_function in checks:
    check_result = check_function(config_text)
```

- checks는 리스트이고, 각 항목은 튜플이다.
- 튜플 안에는 항목 코드, 제목, 실행할 함수가 들어간다.
- check_function(config_text)는 변수에 담긴 함수를 실행하는 문법이다.

9. 보고서 파일 쓰기

점검 결과를 화면에만 출력하면 사라진다. 그래서 reports 폴더를 만들고 Markdown 파일로 저장했다.

```
reports_dir = project_root / "reports"
report_file = reports_dir / "account_management_report.md"

reports_dir.mkdir(exist_ok=True)
```

```
report_file.write_text(report_text, encoding="utf-8")
```

- `mkdir(exist_ok=True)`는 폴더가 이미 있어도 에러를 내지 않는다.
- `write_text()`는 문자열을 파일로 저장한다.
- `encoding='utf-8'`은 한글이 깨지지 않도록 지정한 것이다.

10. 오늘 코드에서 기억할 핵심

개념	한 줄 요약
Path	파일 위치를 안전하게 만들고 읽고 쓰기 위해 사용
리스트	여러 개의 설정 줄이나 결과 문구를 모으기 위해 사용
딕셔너리	code, title, result처럼 이름이 있는 결과 묶음을 만들기 위해 사용
함수	N-01, N-02처럼 점검 로직을 항목별로 분리하기 위해 사용
import	분리한 파일의 함수를 main.py에서 재사용하기 위해 사용
try/except	숫자 변환 실패 같은 예외 상황에서 프로그램이 죽지 않게 하기 위해 사용
코드블록	Markdown에서 Cisco 설정값이 문법으로 오해되지 않게 하기 위해 사용

다음에 N-06을 만들 때도 같은 패턴을 따르면 된다: `n06_cisco.py`를 만들고, `check_n06_cisco()` 함수를 작성한 뒤, `main.py`의 `checks` 리스트에 추가한다.